# An Optimal Algorithm for the Strict Saddlepoint Problem in Non-Square Matrices

Kanhaiya Kumar Jaiswal[a], Kavindra Mohan Dwivedi[a], Varunkumar Jayapaul[a,*]

[a]*School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, Mandi 175005, Himachal Pradesh, India*

**Abstract**

A saddlepoint in a matrix is an entry that is the maximum in its row and simultaneously the minimum in its column. A strict saddlepoint (SSP) of a matrix $\mathcal{M}$ is an entry that is the unique maximum in its row and the unique minimum in its column.

The current best deterministic algorithm solves the SSP problem in $O(n \lg^* n)$ time for an $n \times n$ matrix and in $O(m \lg^* n)$ time for an $m \times n$ matrix, where $m > n$ without loss of generality. Existing techniques for non-square matrices decompose the matrix into several square subproblems and do not exploit the rectangular structure directly.

In this paper, we work directly on $m \times n$ non-square matrices. Our algorithm recursively reduces the number of rows while preserving correctness, by exploiting the asymmetric structure of the input. We show that when $m = \Omega(n \lg n \lg^* n)$, an SSP can be found, or its absence certified, in optimal $O(m + n)$ time in the comparison model.

To the best of our knowledge, this yields the first optimal comparison-based algorithm for the SSP problem on a broad class of sufficiently rectangular matrices.

*Keywords:* strict saddlepoint, saddlepoint, non-square matrices, matrix algorithms, comparison-based algorithms

*Corresponding author

*Email addresses:* s24086@students.iitmandi.ac.in (Kanhaiya Kumar Jaiswal), s24004@students.iitmandi.ac.in (Kavindra Mohan Dwivedi), varunkumar@iitmandi.ac.in (Varunkumar Jayapaul)

## 1. Introduction

### *1.1. Background*

A saddlepoint of a matrix M is an entry that is simultaneously the maximum of its row and the minimum of its column. The notion appears in several areas of mathematics, optimization, and game theory, including the minimax framework of [1]. In this paper we study the algorithmic problem of finding saddlepoints in matrices.

In the context of computer science, the problem of finding a saddlepoint was first mentioned in the classic textbook "The Art of Computer Programming" (1968) by Donald Knuth [2].

A saddlepoint refers to a value rather than a location. If a matrix contains a saddlepoint, then all saddlepoints have the same value, although that value may occur at multiple locations. Not every occurrence of the saddlepoint value is necessarily a saddlepoint.

A strict saddlepoint (SSP) is an entry that is the unique maximum of its row and the unique minimum of its column. Unlike ordinary saddlepoints, an SSP is unique by location whenever it exists. The algorithmic problem studied in this paper is to report the location of an SSP or certify that no SSP exists.

Since finding all the saddlepoints might require one to look at all the entries in the matrix, majority of the literature on saddlepoint focuses on the more interesting problem of finding a strict saddlepoint or finding a single instance of saddlepoint. Most known algorithms try to find a candidate location in matrix which happens to be the saddlepoint by eliminating other location possibilities. Then they verify the value at this candidate location to see it is the maximum in its row and minimum in its column. Thus, these algorithms solve both the problems of finding a saddlepoint and finding the strict saddlepoint if it exists.

### *1.2. Previous Results and Motivation*

The probability that a random $m \times n$ matrix (with i.i.d. continuous entries) has a saddlepoint is $\frac{m+n}{\binom{m+n}{n}}$, as shown by Thorp [3]. This quantity is exponentially small in $\min(m, n)$. Since saddlepoints are exponentially rare in random matrices, efficient algorithms rely on rapidly eliminating rows and columns that cannot contain an SSP.

Llewellyn et al. [4] showed that a strict saddlepoint (SSP) in an $m \times n$ matrix ($m \geq n$) can be found in $O(mn^{\lg_2 3-1})$ time. This was the first result to show that SSP can be found without examining all the entries of a matrix. Another important contribution of this result was to show that one can focus on solving the problem for square matrices and use that technique to the solve the problem on rectangular matrices. They show that if $m > n$ and SSP can be solved in $O(n^{\lg_2 3})$ time in an $n \times n$ matrix, then that same algorithm can be used to solve the SSP problem in $m \times n$ (or $(n \times m)$) matrix $O((m/n)n^{\lg_2 3})$ time. However, this technique is quite generic and not specific to the algorithm they provide.

We re-state one of their lemmas in the following generic form.

**Lemma 1.1.** *[4] If the SSP problem in an $n \times n$ square matrix can be solved in $O(f(n))$ time, then the SSP problem in an $m \times n$ (or $n \times m$) matrix (where $m > n$) can be solved in $O\left(\frac{m}{n} f(n)\right)$ time.*

This lemma ensured that subsequent work on saddlepoint/SSP focused on square matrices.

Next major improvement came few decades later. Bienstock et al. [5] gave an $O(m \lg n)$ deterministic algorithm for finding a strict saddlepoint (SSP) in an $m \times n$ matrix ($m \geq n$), using only $O(m + n)$ queries. Their approach focuses on square matrices and handles rectangular matrices by using the lemma mentioned above. This algorithm was the first algorithm to show that one needs to inspect only $O(n)$ entries of an $O(n \times n)$ matrix to solve the SSP problem. The $O(n \lg n)$ running time is a consequence of the use of a priority queue.

The current best deterministic algorithm is due to Dallant et al. [6], who achieve $O(n \lg^* n)$ time for $n \times n$ matrices using a generalization called *pseudo saddlepoints*. A pseudo saddlepoint is a relaxation of the SSP that can be located efficiently and used to eliminate rows and columns that cannot contain the true SSP. The algorithm iteratively finds pseudo saddlepoints on submatrices, eliminates certified rows and columns, and recurses on a smaller submatrix until the SSP is found or its absence is certified.

They also devise a simple testing mechanism, which given $s$ as a candidate value for saddlepoint in a $m \times n$ matrix, it can verify whether $s$ is its saddlepoint (and SSP) in $O(m+n)$ time. If $s$ is indeed the saddlepoint/SSP, then it will also report its location in the matrix. Previous verification techniques required location in the matrix to verify whether that location contains

the saddlepoint or not. However, this mechanism does not require the location. Since SSP algorithms must also certify absence when no SSP exists, we include in Appendix an explicit $O(m+n)$-size certificate of nonexistence together with a linear-time verification procedure.

**Randomized $O(n)$ expected time.** Randomized algorithms can locate the SSP of an $n \times n$ matrix in $O(n)$ expected time [7]. The key idea is to randomly sample rows and columns to quickly identify promising candidates, then verify. However, no deterministic comparison-based algorithm with linear running time was previously known for rectangular matrices.

**Non-square matrices.** Existing algorithms for rectangular m×n matrices typically reduce the problem to a collection of square n×n subproblems and then apply a square-matrix SSP algorithm to each block. This reduction is captured by Lemma 1.1. Consequently, the current best deterministic algorithm of Dallant et al. [6] yields a running time of $O(m \lg^* n)$ for rectangular matrices. For $m = \omega(n)$, this remains super-linear in the input dimensions. No previous deterministic algorithm has exploited the rectangular structure itself to obtain an $O(m+n)$-time bound.

### *1.3. Our Contribution*

It is known that if the SSP problem on an $n \times n$ square matrix can be solved in $O(n)$ time, then it would imply that SSP problem can be solved in $O(m+n)$ time using Lemma 1.1. We show that even if SSP problem cannot be solved in $O(n)$, it can be shown that SSP problem for sufficiently rectangular matrices can be solved in optimal $O(m+n)$ time.

We introduce a new algorithm, the *Saddlepoint Sieve* (Algorithm 1), that processes any $m \times n$ matrix $\mathcal{M}$ with $m > n$ and reduces it to a submatrix $\mathcal{M}'$ of at most $O(n \lg n)$ rows in $O(m+n)$ time. The sieve inspects at most *one entry per row* and uses a max-priority queue together with $n$ small buffers to maintain a provably correct upper bound on the value of the SSP. This technique is distinct from earlier methods that used decomposition because it does not break $\mathcal{M}$ up into square submatrices. By combining the Saddlepoint Sieve with the result of Dallant et al. [6] applied to $\mathcal{M}'$, we obtain a complete algorithm running in $O(m+n+n \lg n \lg^* n)$ time. When $m = \Omega(n \lg n \lg^* n)$, this reduces to $O(m+n)$, which is optimal in the comparison model. Since verifying a candidate SSP or a certificate of absence requires $\Omega(m+n)$ information, no asymptotically faster comparison-based algorithm is possible.

## 2. Basic Properties of Strict Saddlepoints

We present several structural properties of SSPs that are used throughout the paper.

**Lemma 2.1.** *If $\mathcal{D}$ contains an entry from each column of $\mathcal{M}$ and* $\max(\mathcal{D})$ *is the largest entry in $\mathcal{D}$, then any row containing a value greater than or equal to* $\max(\mathcal{D})$ *does not contain the strict saddlepoint of $\mathcal{M}$.*

*Proof.* Suppose row $r$ contains a value $v$ such that $v \geq \max(\mathcal{D})$. Since $\mathcal{D}$ contains an entry from every column, every column contains a value at most $\max(\mathcal{D})$. Therefore:

- if $v > \max(\mathcal{D})$, then $v$ cannot be the minimum of its column;
- if $v = \max(\mathcal{D})$, then either $v$ is not the minimum of its column, or it is not the unique minimum of its column.

Hence $v$ cannot be an SSP.

Now consider any other entry $v'$ in row $r$. If $v' \leq v$, then $v'$ is not the unique maximum of row $r$. If $v' > v$, then $v'$ cannot be the minimum of its column, since $v \geq \max(\mathcal{D})$.

Therefore no entry in row $r$ can be an SSP. □

Although this observation is true for square and non-square matrices, this lemma is especially useful in case of non-square matrices where the number of rows is larger than the number of columns.

Analogously, one can also prove

**Lemma 2.2.** *If $\mathcal{D}$ contains an entry from each row of $\mathcal{M}$ and* $\min(\mathcal{D})$ *is the smallest entry in $\mathcal{D}$, then any column containing a value smaller than or equal to* $\min(\mathcal{D})$ *does not contain the strict saddlepoint of $\mathcal{M}$.*

The following lemmas allow us to use decrease and conquer technique to solve the SSP problem.

**Lemma 2.3.** *Suppose row $r$ of the matrix $\mathcal{M}$ does not contain the SSP and $\mathcal{M}'$ denotes the matrix which is obtained by discarding row $r$ of $\mathcal{M}$. If $\mathcal{M}$ has an SSP, then $\mathcal{M}'$ also has an SSP and the value of SSP of $\mathcal{M}$ is equal to the value of SSP of $\mathcal{M}'$.*

*Proof.* Suppose we somehow know that the row $r$ of $\mathcal{M}$ does not contain the SSP. If $\mathcal{M}$ has an SSP, then let it be present at location $[r'][c]$ where row $r' \neq r$. Since the SSP is the unique maximum of its row in $\mathcal{M}$, it will continue to be the unique maximum in its corresponding row in $\mathcal{M}'$. Column $c$ in $\mathcal{M}'$ has one less entry compared to column $c$ in $\mathcal{M}$. However, that missing entry is not the SSP, since the row which was discarded does not contain the SSP and thus the unique minimum of column $c$ in $\mathcal{M}$ is also the unique minimum of column $c$ in $\mathcal{M}'$. □

The previous lemma formalizes the decrease-and-conquer principle underlying many SSP algorithms.

Analogously, one can also prove

**Lemma 2.4.** *Suppose column $c$ of the matrix $\mathcal{M}$ does not contain the SSP and $\mathcal{M}'$ denotes the matrix which is obtained by discarding column $c$ of $\mathcal{M}$. If $\mathcal{M}$ has an SSP, then $\mathcal{M}'$ also has an SSP and the value of SSP of $\mathcal{M}$ is equal to the value of SSP of $\mathcal{M}'$.*

*Remark* 2.5. The converse of the lemmas does not hold. If $\mathcal{M}$ does not contain an SSP, then discarding a row $r$ from $\mathcal{M}$ to obtain $\mathcal{M}'$ need not ensure that $\mathcal{M}'$ also does not have an SSP. There is a possibility that $\mathcal{M}$ has no SSP, but $\mathcal{M}'$ does have an SSP. For example, the matrix $\left[\begin{smallmatrix} 1 & 3 \\ 4 & 2 \end{smallmatrix}\right]$ does not contain an SSP, but removing any row creates an SSP in the remaining submatrix.

Thus repeated application of lemmas 2.3 and/or 2.4 can possibly introduce an SSP in the resultant smaller matrix. However, once a smaller matrix has an SSP with value $s$, repeated application of these lemmas, will not change the SSP's value. If one uses the two aforementioned lemmas and finds a value $s$ to be the candidate for SSP, one needs to perform a final verification by checking whether $s$ is truly the SSP of original matrix or not.

This can be done efficiently using the following lemma

**Lemma 2.6.** *[6] Given an $m \times n$ matrix $\mathcal{M}$ and a value $s$, we can determine in time $O(m+n)$ whether $s$ is the value of the SSP and if so return its location.*

In the following section, we will focus on finding the value of the SSP. Once the value of the SSP is found the preceeding lemma can help us find its location in the matrix.

## 3. Core Algorithmic Results

We present the two phases of our algorithm. Section 3.1 gives the Strict Saddlepoint Sieve. Section 3.2 describes the application of the faster-than-sorting algorithm to the reduced matrix.

Lemma 2.1 + Lemma 2.3 are the two ingredients behind the sieve.

**Algorithm 1:** Strict Saddlepoint-Sieve for $\mathcal{M}_{m\times n}$

**Input:** $m \times n$ matrix $\mathcal{M}$ with $m \geq n$

**Output:** Matrix $\mathcal{M}'$ containing the row with the SSP of $\mathcal{M}$, if it exists

```
Set up a max-priority queue 𝒟 with diagonal elements of the top n rows of ℳ;
for i ∈ [n] do
    S_i ← ∅;
while there exist uninspected rows do
    Let j be the column containing the maximum element in 𝒟;
    S_j ← ∅;
    Inspect lg n uninspected rows and extract values from column j corresponding to the column of max(𝒟) and keep them in S_j;
    if max(𝒟) > min(S_j) then
        Delete maximum element of D and insert it in S_j;
        Delete minimum element of S_j and insert it in 𝒟;
    else if max(𝒟) ≤ min(S_j) then
        S_j ← ∅;
return ℳ' which contains only those rows in ℳ which have a representative in 𝒟 or ∪_{i=1}^{n} S_i;
```

The priority queue $\mathcal{D}$ and the sets $S_i$ store matrix entries together with their row indices as auxiliary information. The algorithm is described for matrices with $m \geq n$. An analogous algorithm applies when $m < n$. Although the algorithm is valid for all $m \geq n$, it is useful only when $m = \Omega(n \lg n)$.

### *3.1. Phase 1 — Strict Saddlepoint Sieve for $\mathcal{M}_{m\times n}$*

**Theorem 3.1.** *Given an $m \times n$ matrix $\mathcal{M}$, there exists an algorithm which runs in $O(m + n)$ time and returns an $m' \times n$ submatrix $\mathcal{M}'$ (where $m' = O(n \lg n)$)such that the value of SSP of $\mathcal{M}$ (if it exists) is also the value of SSP of $\mathcal{M}'$.*

*Proof.* First we prove the correctness of the algorithm. The algorithm never discards any column. When the algorithm set $S_j$ to null, it discards rows whose representatives are present in $S_j$. This happens only at lines 6 and 12. Therefore it suffices to show that every discarded row cannot contain an SSP. At line 6, if $S_j$ is not null, the conditions at lines 5 guarantee that every row in $S_j$ contains an entry in column $j$ whose value is at least $\max(\mathcal{D})$. Therefore, by Lemma 2.1, none of these rows whose representative is in $S_j$ can contain an SSP. An identical argument applies to the rows discarded at line 12, due to the condition checked at line 11. Since every discarded row is certified not to contain an SSP, Lemma 2.3 implies that if $\mathcal{M}$ contains an SSP, then $\mathcal{M}'$ also contains an SSP of the same value.

We analyze the bounds on the size of the output. The number of columns in $\mathcal{M}$ and $\mathcal{M}'$ remain unchanged, since the algorithm does not discard any columns. By construction, each set $S_i$ contains at most $\lg n$ rows. The set $\mathcal{D}$ contains exactly one representative entry from each column.

Each probe is performed on a previously uninspected row; therefore no row contributes more than one representative entry to $\mathcal{D}$. Hence the rows represented by $\mathcal{D}$ contribute at most $n$ rows to the output. Since each set $S_i$ contains at most $\lg n$ rows and there are $n$ such sets, the total number of rows contributed by the sets $S_i$ is at most $n \lg n$. Therefore, $\mathcal{M}'$ contains at most $n + n \lg n = O(n \lg n)$ rows.

We analyze the time complexity of the algorithm 1. The algorithm probes exactly one entry per row. This probing is done at lines 1 and 7 of the pseudocode. Thus, $m$ entries are inspected over the entire execution. The time spent initializing the priority queue $\mathcal{D}$ at line 1 is $O(n)$. Then, after probing $\lg n$ entries and adding them to set $S_j$, we perform a Delete maximum operation on $\mathcal{D}$ and track the minimum of $S_j$ (which has $\lg n$ elements) in $O(\lg n)$ time. Since each iteration processes $\lg n$ previously uninspected rows, the number of iterations is $O(m/\lg n)$. Therefore the total time spent inside the loop is $O\left(\frac{m}{\lg n} \cdot \lg n\right) = O(m)$. Thus, the while loop requires $O(m)$ time in total. Including the initialization of $\mathcal{D}$, the overall running time of the algorithm is $O(m + n)$.

□

The rows discarded during Phase 1 are eliminated using explicit witness entries. Together with the entries maintained in $\mathcal{D}$, these witnesses can be used to construct a certificate of nonexistence as described in the appendix.

Theorem 3.1 reduces the SSP problem on an $m \times n$ matrix to an SSP problem on an $m' \times n$ matrix with $m' = O(n \lg n)$ in $O(m+n)$ time. We now combine this reduction with existing SSP algorithms.

*3.2. Phase 2 — Faster-than-Sorting on the Reduced Matrix*

After Phase 1, the matrix is reduced to $\mathcal{M}'$ with $O(n \lg n)$ rows and $n$ columns. Assuming $\mathcal{M}$ has an SSP, since the value of SSP of $\mathcal{M}$ remains the value of SSP of $\mathcal{M}'$, we can use other existing algorithms to find the value of SSP of $\mathcal{M}'$.

Existing deterministic algorithms for rectangular matrices reduce the problem to a collection of square $n \times n$ subproblems and then apply an SSP algorithm for square matrices. Consequently, once Phase 1 has reduced the matrix to $O(n \lg n) \times n$, any existing algorithm for square matrices can be used as a black box via Lemma 1.1.

We consider two deterministic algorithms for solving the SSP problem on $\mathcal{M}'$. Although neither algorithm explicitly outputs a certificate of nonexistence, such a certificate can be reconstructed from the entries probed during execution.

***Case(a):*** *Using Bienstock et al.[5].* Bienstock et al. [5] solve the SSP problem on an $n \times n$ matrix in $O(n \lg n)$ time. Applying Lemma 1.1 to the $O(n \lg n) \times n$ matrix $\mathcal{M}'$ yields a running time of $O(n \lg^2 n)$.

$$T(m,n) \;=\; \underbrace{O(m+n)}_{\text{Phase 1}} + \underbrace{O(n \lg^2 n)}_{\text{Phase 2: } \textbf{Case (a)}} \;=\; O\big(m+n+n \lg^2 n\big)\,.$$

In particular, when $m = \Omega(n \lg^2 n)$, the overall running time simplifies to $O(m+n)$. Although this is not the strongest result, it already yields an optimal $O(m+n)$ algorithm whenever $m = \Omega(n \lg^2 n)$.

***Case(b):*** *Using Dallant et al. [6].* Dallant et al. [6] solve the SSP problem on an $n \times n$ matrix in $O(n \lg^* n)$ time. Applying Lemma 1.1 to $\mathcal{M}'$ gives a running time of $O(n \lg n \lg^* n)$.

**Overall complexity**

$$T(m,n) \;=\; \underbrace{O(m+n)}_{\text{Phase 1}} + \underbrace{O(n \lg n \lg^* n)}_{\text{Phase 2: } \textbf{Case (b)}} \;=\; O(m+n+n \lg n \lg^* n)\,.$$

In particular, when $m = \Omega(n \lg n \lg^* n)$, the overall running time becomes $O(m+n)$. This improves upon the previous best deterministic bound of

$O(m \lg^* n)$ and establishes optimal linear-time complexity in the comparison model for this class of rectangular matrices.

Combining the Phase 1 reduction with the running time of Case (b), we obtain our main algorithmic result.

**Theorem 3.2.** *The SSP problem on an $m \times n$ matrix can be solved in*

$$O(m + n + n \lg n \lg^* n)$$

*deterministic time.*

**Corollary 3.3.** *When*

$$m = \Omega(n \lg n \lg^* n),$$

*the SSP problem can be solved in optimal $O(m + n)$ time in the comparison model.*

## 4. Conclusion and Future Work

We presented an optimal $O(m + n)$ comparison-based algorithm for the strict saddlepoint (SSP) problem on $m \times n$ matrices satisfying $m = \Omega(n \lg n \lg^* n)$. Unlike previous approaches, which solve the non-square problem by reducing it to a collection of square subproblems, our algorithm operates directly on rectangular matrices and exploits their structure through a sieve. This demonstrates that the rectangular configuration can be leveraged to obtain asymptotically faster algorithms than those obtained through the standard reduction to square matrices.

Several questions remain open. Our sieve yields an improvement when $m = \Omega(n \lg n \lg^* n)$, but provides no asymptotic advantage for the regime

$$n < m < n \lg n \lg^* n.$$

An important direction for future work is to determine whether the running time can be improved for matrices in this range.

More broadly, previous work on the SSP problem has largely followed the strategy of designing algorithms for square matrices and extending them to rectangular matrices. Our results suggest a complementary research direction: transforming square instances into suitably rectangular instances and then exploiting algorithms designed specifically for non-square matrices. It would be interesting to investigate whether such reductions can lead to improved algorithms for the square-matrix case as well.

## References


[1] J. von Neumann, O. Morgenstern, A. Rubinstein, Theory of Games and Economic Behavior: 60th Anniversary Commemorative Edition, Princeton University Press, 1944.

[2] D. E. Knuth, The Art of Computer Programming, Volume 1: Fundamental Algorithms, 1st Edition, Addison-Wesley Publishing Company, Reading, Massachusetts, 1968.

[3] E. O. Thorp, The probability that a matrix has a saddle point, Information Sciences 19 (2) (1979) 91–95.

[4] D. C. Llewellyn, C. Tovey, M. Trick, Finding saddlepoints of two-person, zero sum games, The American Mathematical Monthly 95 (10) (1988) 912–918.

[5] D. Bienstock, F. Chung, M. L. Fredman, A. A. Schäffer, P. W. Shor, S. Suri, A note on finding a strict saddlepoint, The American Mathematical Monthly 98 (5) (1991) 418–419.

[6] J. Dallant, F. Haagensen, R. Jacob, L. Kozma, S. Wild, Finding the saddlepoint faster than sorting, in: M. Parter, S. Pettie (Eds.), 2024 Symposium on Simplicity in Algorithms, SOSA 2024, Alexandria, VA, USA, January 8-10, 2024, SIAM, 2024, pp. 168–178.

[7] J. Dallant, K. G. L. Haagensen, R. Jacob, L. Kozma, S. Wild, An optimal randomized algorithm for finding the saddlepoint, in: 32nd Annual European Symposium on Algorithms (ESA 2024), Vol. 308 of LIPIcs, 2024, pp. 44:1–44:16.


## Appendix A. Certificate for Absence of a Saddlepoint or SSP

Many saddlepoint and SSP algorithms must not only locate a saddlepoint when one exists, but also certify its absence when no saddlepoint is present. In this appendix, we describe a simple certificate of nonexistence.

**Lemma Appendix A.1.** *Let $\mathcal{M}$ be a matrix and let $v$ be a value. Suppose every row of $\mathcal{M}$ contains an entry greater than $v$, and every column of $\mathcal{M}$ contains an entry smaller than $v$. Then $\mathcal{M}$ contains no saddlepoint and hence no SSP.*

*Proof.* Since every row contains an entry greater than $v$, the maximum entry of every row is greater than $v$. Therefore a saddlepoint, if it exists, must have value greater than $v$. Similarly, since every column contains an entry smaller than $v$, the minimum entry of every column is smaller than $v$. Therefore a saddlepoint, if it exists, must have value smaller than $v$.

No matrix entry can be simultaneously greater than and smaller than $v$. Hence $\mathcal{M}$ contains no saddlepoint. Since every SSP is a saddlepoint, $\mathcal{M}$ cannot contain an SSP either. □

Based on this lemma, one can define a certificate of absence of saddlepoint or SSP as follows.

Let $R$ contain one location from each row of $\mathcal{M}$ and let $C$ contain one location from each column of $\mathcal{M}$.

$$r = \min_{(i,j)\in R} \mathcal{M}[i][j] \qquad \text{and} \qquad c = \max_{(i,j)\in C} \mathcal{M}[i][j].$$

If $r > c$, then $\mathcal{M}$ contains no saddlepoint and hence no SSP. In this case, the pair $(R, C)$ forms a certificate of nonexistence.

The following matrix illustrates the certificate. $R = \{(1,2),(2,4),(3,4),(4,1)\}$ whereas $C = \{(3,1),(4,2),(1,3),(1,4)\}$. For this example, $r = 10$ and $c = 5$, and therefore $r > c$.

Hence the matrix contains no saddlepoint. The condition $r > c$ implies that every row contains an entry greater than $c$ and every column contains an entry smaller than $r$. Applying the lemma with any value $v$ satisfying $c < v < r$ proves that no saddlepoint can exist.

$$\begin{bmatrix} 1 & 10 & 4 & 2 \\ 8 & 9 & 16 & 12 \\ 3 & 7 & 15 & 13 \\ 11 & 5 & 6 & 14 \end{bmatrix}$$

Although this certificate is not stated explicitly in prior work, it can be extracted from the information gathered by many SSP algorithms. Algorithms that record the witnesses used to eliminate rows and columns naturally produce such a certificate. For an $m \times n$ matrix, the certificate has size $O(m + n)$, which is asymptotically optimal. Moreover, its validity can be verified in optimal $O(m + n)$ time.